Value Iteration Residual Network with Self-attention

Jinyu Cai(✉), Jialong Li, Zhenyu Mao, and Kenji Tei

Waseda University, Tokyo, Japan
bluelink@toki.waseda.jp, lijialong@fuji.waseda.jp

Abstract. The Value Iteration Network (VIN) is a neural network widely used in path-finding reinforcement learning problems. The planning module in VIN enables the network to understand the nature of a problem, thus giving the network an impressive generalization ability. However, reinforcement learning (RL) with VIN can not guarantee efficient training due to the network depth and max-pooling operation. A great network depth makes it harder for the network to learn from samples when using gradient descent algorithms. The max-pooling operation may increase the difficulty of learning negative rewards due to overestimation. This paper proposes a new neural network, Value Iteration Residual Network (VIRN) with Self-Attention, using a unique spatial self-attention module and aggressive iteration to solve the above-mentioned problems. A preliminary evaluation using Mr. Pac-Man demonstrated that VIRN effectively improved the training efficiency compared with VIN.

Keywords: Deep Reinforcement Learning · Value Iteration · Self-Attention · Shortcut Connection

1 Introduction

RL is an increasingly popular group of machine learning algorithms [1], where an RL agent autonomously learns an optimal policy by searching in the state-action space and interacting with the environment [2]. Most RL algorithms with neural networks (i.e., deep reinforcement learning) use reactive networks, such as Multilayer Perceptron (MLP), Convolutional Neural Networks (CNN) [3], and Long Short-Term Memory (LSTM) [4]. For these networks, it is difficult to say whether they understand a behavior's goal-directed nature (e.g., avoid obstacles and reach the destination in path-finding problems) [5]. As a result, they may not guarantee good performance when the environment is changed (e.g., a different map is given in path-finding problems).

The Value Iteration Network (VIN) [5] is a neural network first proposed to help solve path-finding problems. The key component in VIN is the Value Iteration (VI) module, which helps the network understand the nature of a problem. The VI module first calculates how each action affects the environment

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023
A. Abraham et al. (Eds.): ISDA 2022, LNNS 716, pp. 16–24, 2023.
https://doi.org/10.1007/978-3-031-35501-1_2

using convolutional iterative computation and a special max-pooling operation, and it then selects the optimal action as output. The VI module enables the network to understand the problem and learn how to plan [6]; thus, the network performances well even in dynamic environments.

However, training efficiency in RL with VIN is not guaranteed due to two reasons:

- Current studies on VIN often deal with a large input size but use a small kernel for convolution [5,7,8], which leads to a great network depth for the VI module. Such a great depth makes it more difficult for the network to learn from samples when applying gradient descent algorithms [9].
- The max-pooling operation in the VI module may cause the impact of negative rewards to be harder to spread, as it will be easily overwritten by that of far-range positive rewards. The resulting overestimation makes the negative reward information unable to be effectively learned.

To solve the above-mentioned problems, this paper proposes the Value Iteration Residual Network (VIRN) to increase the RL training efficiency. The key idea of VIRN is to reduce the network depth while reducing its negative impact on training efficiency, and to save the results of each iteration to the final output before overwriting. The main contributions of this paper are as follows

- This paper proposes aggressive iteration(i.e., using larger convolution kernels with shorter iterations) to reduce the network depth.
- This paper proposes a self-attention module for relating the output of each iteration directly to the final output, which effectively alleviates the overwriting problem and reduces the negative impact of network depth.

The rest of this paper is organized as follows. Section 2 introduces background techniques. Section 3 introduces related work and Sect. 4 explains VIRN. Section 5 introduces preliminary evaluation. Section 6 concludes this paper and introduces future work.

2 Background

2.1 Reinforcement Learning

RL is a branch of machine learning that cope with learning policies for sequential decision-making problems. An RL problem is typically formulated in terms of computing the optimal policies for a Markov Decision Process (MDP) [10]. Formally, RL can be represented as a MDP, which consists of a tuple [S, A, R, P], with S being the set of state s, A being the set of action a, R being a reward function $R(s,a)$, and P being transition probabilities $P(s'|s,a)$. In RL, the agent's objective is to learn a policy π that maximizes the cumulative expected reward. A standard method for finding π is to iteratively compute the value function:

$$\begin{aligned} &V_{k+1}(s) = max_a Q_k(s,a) \ \ \forall s, \\ &where\ Q_k(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) V_k(s') \end{aligned} \tag{1}$$

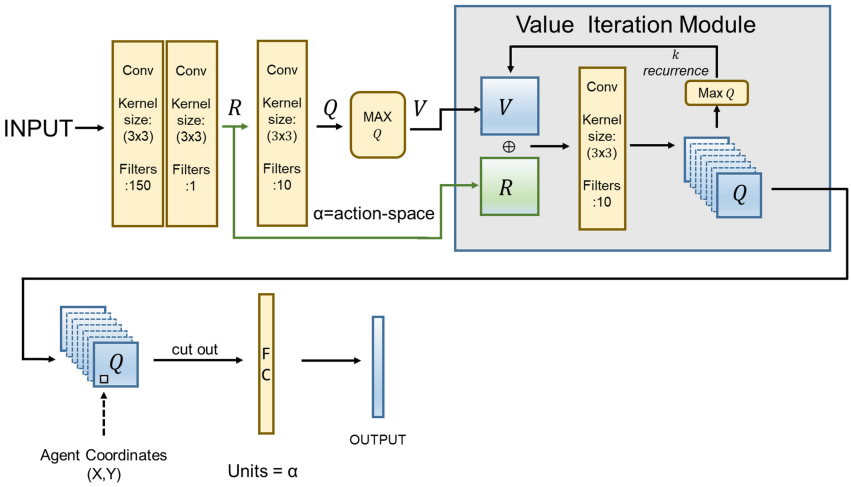


Fig. 1. Overview of Value Iteration Network

where $Q_k(s,a)$ is the value function of action a in state s at n iterations, and $\gamma \in (0,1]$ is the discount rate updating it until convergence.

2.2 Value Iteration Network

VIN is a neural network with a planning module to help solve the path-finding problem. The core idea of the VIN, as shown in Fig. 1, is to use a neural network to express the value iteration algorithm[1]. Taking a path-finding problem as an example, the input of the network is the observed map information states. VIN will first pass through two layers of CNN to generate a reward map. Then the reward map, together with an initialized value map of the same size, is input into the VI module. During the value iteration, the reward map and the value map are input into the convolutional layer, which calculates an action value map(Q map) as an output. Then, the Q map will be maximized on the n-dimension, and the value map and the reward map get updated. The above-mentioned procedure repeats until the value map converges. Finally, the fully connected layer uses values of the agent's position on the Q map to calculate action values as final outputs.

3 Related Work

VIN on Multiple Levels of Abstraction [11] deals with large-size and high-dimension domains by sampling the large-size domains into multiple levels of features. The information loss caused by the decreased resolution is compensated for by increasing the number of feature representations; thus, the capability of VIN is improved. VIN on Multiple Levels of Abstraction manages to deal with

large-size domains through a different approach, but the overestimation problem remains.

The Generalized Value Iteration Network (GVIN) was proposed to deal with irregular spatial graphs inputs [12]. GVIN replaces the traditional convolution operator with a novel graph convolution operator, which enables GVIN to learn and plan on irregular spatial graphs. Furthermore, GVIN uses a novel episodic Q-learning, and it is more stable than traditional n-step Q-learning. GVIN aims to increase training stability, while the proposal in this paper focuses on increasing training efficiency.

Value Iteration Networks with Double Estimator for Planetary Rover Path Planning (dVIN) [8] was also proposed to solve the overestimation problem but uses a double estimator method. dVIN uses a double convolutional layer instead of one in the VI module, and it uses the weighted mean of the two values in order to approximate the maximum expected value of the next iteration. This idea is similar to [13], which has been demonstrated to be able to reduce overestimation and result in more stability. dVIN alleviates the overestimation problem to some extent, but the error accumulation due to iterations still remains.

4 Value Iteration Residual Network

4.1 Overview

As shown in Fig. 2, compared with VIN, there is an additional self-attention module in VIRN. The overall workflow of VIRN is as follows: pixel information is first input into CNN. The reward map returned by CNN and an initialized value map are conveyed to the VI module. The VI module returns the value map after each iteration to the self-attention module generating an integrated value map. Finally, the final convolutional layer outputs the action values on the basis of the agent coordinate.

4.2 VI Module

The core idea of the VI module is to use large convolution kernels to reduce the number of iterations and the network depth. The convolution kernels used in the VI module in the original VIN are 3×3 in size. Using small convolution kernels means that more iterations and deeper networks are needed to make the value function converge. VIRN actively uses larger convolution kernels in accordance with the different input map sizes. A larger convolution kernel also makes the computation more paralleled, thus further improving training efficiency.

In addition, the number of convolution kernels used in VIRN is different from VIN. In the original VI module, the number of kernels q is larger than the size of the action space; thus, decoding is required to convert q-dimension vectors into action values. VIRN directly uses the size of the action space as the number of the convolutional kernels in the VI module, skipping the decoding procedure and thus reducing encoding-decoding loss.

In VIRN, the value map after each iteration is also returned to the self-attention module. Since each iteration may cause an overwriting problem, as the number of iterations increases, more information might be lost. Returning the result of each iteration to the self-attention module provides the network with more useful information, enabling the self-attention modules to generate an integrated value map.

4.3 Self-attention Module

In VIRN, the future is not always taken into account when planning. For example, when we are playing a game, if we face a trap that will lead to death, we always back off and look for another way out, no matter how attractive the reward behind the trap might be.

$$V' = \sum_{i \in k} \left(\frac{\exp(\alpha_i)}{\sum_{j \in k} \exp(\alpha_j)} \cdot v_i \right) \qquad (2)$$
$$\alpha_i = r w^r \cdot v_i w^v$$

The reward map returned from the CNN goes through a feature extraction process and is input into the self-attention module with the output of the VI module. The self-attention module first multiplies the result of each iteration in the VI module and the feature of the reward map by the weight matrix w^v, w^r. The dot product of the two previous results is correlation α. Then, the probability distribution of the k correlations is calculated by a softmax function. Finally, the integrated value map is obtained by calculating the weighted average of k value maps (Eq. 2).

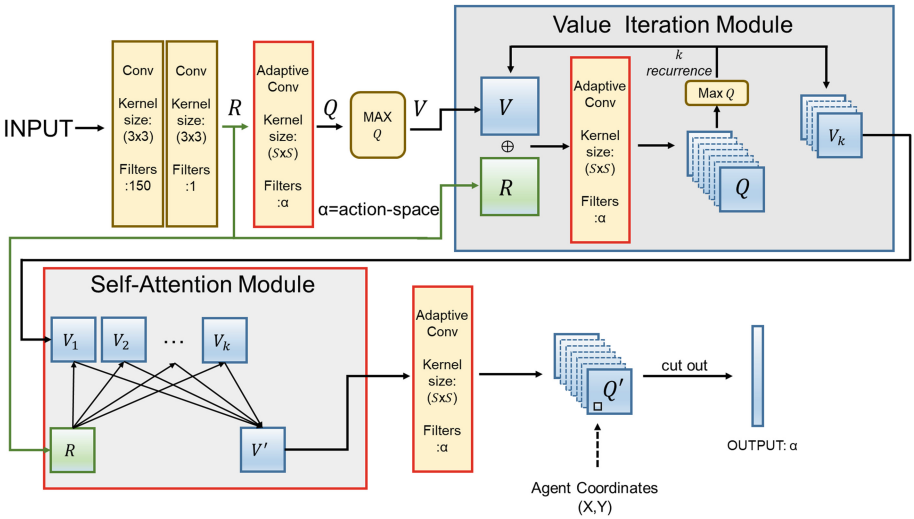


Fig. 2. Overview of Value Iteration Residual Network with Self-attention

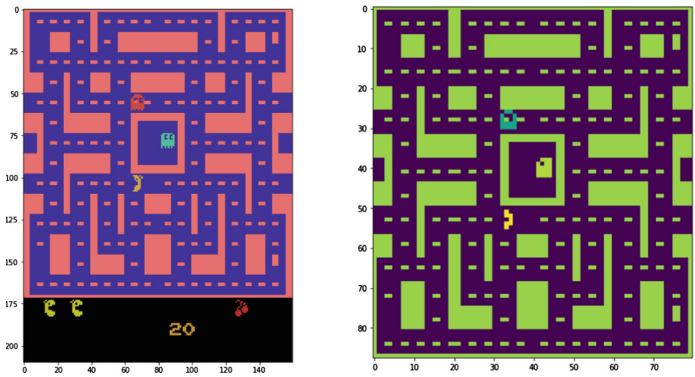
By training the weights w^v, w^r, the self-attention module enables the network to choose the result of iteration autonomously. This procedure is similar to the residual connection [9], which also effectively alleviates the problem of gradient explosion and gradient disappearance, making it easier for the network to learn from samples.

5 Preliminary Evaluation

In this section, the experiment settings are first explained, and then the results are shown with a brief discussion. A preliminary evaluation was conducted using Mr. Pac-Man under the research question of ***how well does VIRN increase the training efficiency compared with VIN***? For systematical evaluation, the learning speed is considered, shown as the change in average scores in accordance with the number of training episodes and training time (Fig. 3).

5.1 Experiment Settings

The Mr. Pac-Man game is actually very similar to the 2D path-finding problem but with a more complex environment. For example, the moving paths of the enemy and the scoring criteria change after specific events in the game. The performance difference in handling complex environments between different networks is more obvious. To reduce training costs, the map used for the experiment had a size of 84*80*1 instead of the original 210*180*3 in the game. The enemy's movement information is recorded every four frames, so the input data was at a size of 84*80*4.



(a) Nature Image (b) Input Image

Fig. 3. Mr. Pacman game image

For the convenience of illustration, the evaluation also featured IVIN, which has the same structure as VIN except for the iteration in the VI module. VIRN

and IVIN used an 11 × 11 convolution kernel for 15 iterations, and VIN used a 3 × 3 convolution kernel for 100 iterations.

$$\begin{aligned} targetQ &= r + \gamma * Qmax_{a'}(s', a') \\ loss &= E[(targetQ - Q(s_t, a_t))]^2 \end{aligned} \tag{3}$$

Deep Q-learning was used as the baseline training method, where the learning rate was 0.0001, reward decay was 0.99, epsilon greedy was 0.05, and the target network altered every episode. Mini-batch optimization was also used, where the batch size was 32. The loss function was defined as Eq. 3.

5.2 Experiment Result

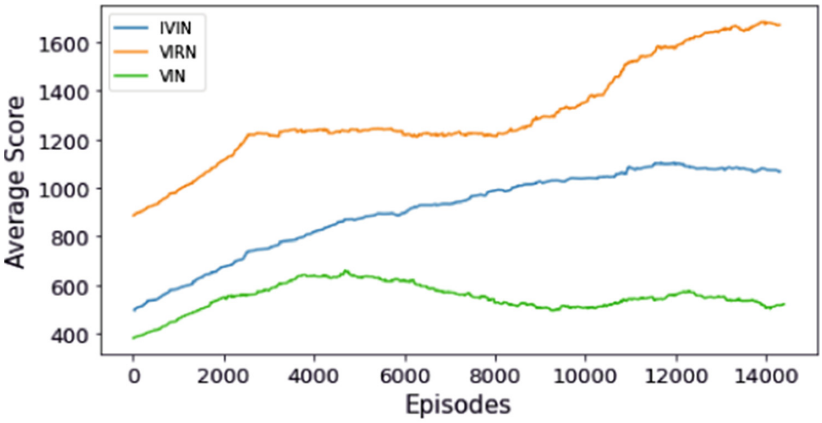
Figure 4a shows that VIRN had obvious advantages compared with VIN and IVIN. The average score of VIN peaked at about 680 points around 5,000 episodes, after which the scoring curve converged. IVIN outperformed VIN from the early stage of training and saved about 80% of training episodes to reach the peak of the VIN curve. VIRN had the highest training efficiency from the early stage of training, and its performance surpassed the highest score of IVIN at about 2,000 episodes, which also saved about 80% of training episodes. The highest average scores of VIRN, IVIN, and VIN within 15,000 episodes were around 1700, 1100, and 680 points, respectively. When the curve of VIRN still showed an increasing trend at the end of the experiment (15,000 episodes), the curves of VIN and IVIN had already reached convergence.

On the other hand, in the time-score comparison shown in Fig. 4b, VIRN and IVIN also had advantages compared with VIN. In particular, IVIN had an extremely fast computing speed. It only took IVIN 100,000 s to complete 15,000 episodes of training. Although the average score of both VIRN and IVIN increased rapidly in the starting 50,000 s, the total training time with VIRN was much longer than IVIN. VIRN is not the fastest method, but its performance within the same training time is always the best.

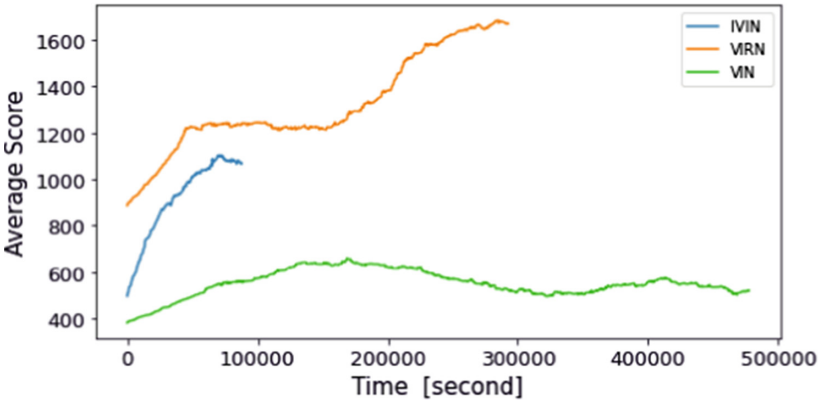
5.3 Discussion

There are two main improvements in VIRN compared with VIN: aggressive iteration and the self-attention module. As for the 84*80*4 input, VIN was too deep due to the higher number of iterations, leading to a lower training efficiency. Also, because of the depth, some weights in the network could not be trained effectively; thus, the final performance of the network (i.e., the highest score) was worse. Comparing VIN with IVIN, it can be concluded that aggressive iteration can effectively reduce the depth of the network and improve the training efficiency. In addition, aggressive iteration leads to more parallel computing and significantly reduces computation time.

During the early stage of training, the performance of VIRN was the best (note that the initial performances of the three methods were all around 300). This is attributed to the residual connection-like structure of the self-attention



(a) Average Score v.s. Episodes



(b) Average Score v.s. Time

Fig. 4. Comparison of VIRN, IVIN, and VIN in Mr. Pac-Man

module in VIRN enabling the network to be trained more efficiently. On the other hand, the self-attention module improved the network's ability to handle complex environments, which lead to better performance in this experiment. Although the self-attention module increased the computation time, VIRN still performed better than VIN.

Though in this experiment VIRN had a significant improvement in training efficiency compared with VIN, we can reasonably infer that if the input size exceeds a certain level, a part of the weights in the VIRN cannot be trained effectively, so the performance of the network will be degraded. Last but not least, the experiment settings used in the evaluation were fixed, so it will take more experiments with different settings and environments to further evaluate the performance of VIRN.

6 Conclusion

This paper proposed a new neural network, VIRN, to increase the training efficiency in RL. Compared with VIN, VIRN has a self-attention module and more aggressive iteration, alleviating the overwriting problem and effectively reducing the network depth. A preliminary evaluation showed that VIRN significantly improved the network training efficiency and outperformed VIN and IVIN in Mr. Pac-Man.

However, VIRN needs to be improved in the following aspects. The first is to shorten the computation time, optimizing the change in computation time in accordance with the input size from exponential to linear. Second, to better evaluate VIRN, more experiments with different settings and environments are required.

Acknowledgement. This work was partially supported by JSPS KAKENHI, JSPS Research Fellowships for Young Scientists.

References

1. Arulkumaran, K., Peter Deisenroth, M., Brundage, M., Anthony Bharath, A.: Deep reinforcement learning: a brief survey. IEEE Signal Process. Mag. **34**, 26–38 (2017)
2. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. A Bradford Book. The MIT Press, Cambridge, MA, USA (2018)
3. Krizhevsky, A., Sutskever, I., Hinton, G.E.: ImageNet classification with deep convolutional neural networks. Commun. ACM **60**(6), 84–90 (2017)
4. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Comput. **9**(8), 1735–1780 (1997)
5. Tamar, A., Wu, Y., Thomas, G., Levine, S., Abbeel, P.: Value iteration networks. In: Lee, D., Sugiyama, M., et al., (eds) Advances in Neural Information Processing Systems, vol. 29. Curran Associates, Inc., (2016)
6. Bertsekas, D.: Dynamic programming and optimal control, vol. 1. Athena Scientific (2012)
7. Shen, J., Hankui Zhuo, H., Xu, J., Zhong, B., Jialin Pan, S.: Transfer value iteration networks. In: The Thirty-Fourth AAAI Conference on Artificial Intelligence, pp. 5676–5683. AAAI Press (2020)
8. Jin, X., Lan, W., Wang, T., Yu, P.: Value iteration networks with double estimator for planetary rover path planning. Sensors (Basel, Switzerland), 21 (2021)
9. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778 (2016)
10. Bellman, R.: A markovian decision process. J. Math. Mech. **6**(5), 679–684 (1957)
11. Schleich, D., Klamt, T., Behnke, S.: Value iteration networks on multiple levels of abstraction. Robotics: Science and Systems XV, abs/1905.11068 (2019)
12. Niu, S., Chen, S., Guo, H., et al.: Generalized value iteration networks: life beyond lattices. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, (2018)
13. Hasselt, H.: Double q-learning. In: Advances in Neural Information Processing Systems, vol. 23, (2010)